

Laboratório de SOC integrado

Fase 1 — Fundação do ambiente e implantação do service desk

Campo	Valor
Autor	Kauan Trinchetti
Projeto	Wazuh (SIEM) + Zabbix (monitoramento) + GLPI (service desk)
Fase	1 de 7 — Fundação e Service Desk
Ambiente	VM Ubuntu Server 24.04 · 5 GB RAM · 2 vCPU · Docker Engine
Status	Concluída

1. Objetivo

Construir um laboratório de operações de segurança no qual as três camadas — detecção, monitoramento e atendimento — operem de forma integrada, com alertas críticos gerando chamados automaticamente no service desk.

A Fase 1 estabelece a fundação do ambiente e implanta o GLPI, que atuará como destino das integrações nas fases seguintes. A escolha de começar pelo service desk é deliberada: ele é o componente mais simples de operar e o pré-requisito das integrações.

2. Restrições e decisões de arquitetura

O laboratório foi construído sobre um host com 8 GB de RAM total, abaixo do recomendado pela documentação do Wazuh. Isso exigiu decisões explícitas de dimensionamento:

Decisão	Justificativa	Ganho
VM única de 5 GB, headless	Elimina o consumo da interface gráfica; acesso por SSH	~1 GB
Instância MariaDB compartilhada	GLPI e Zabbix em bancos distintos, um único processo	~350 MB
Heap do OpenSearch limitado a 512m	O Wazuh indexer é o maior consumidor da stack	~500 MB
mem_limit por container	Impede que um serviço comprometa os demais	estabilidade
Estação Windows como endpoint	Dispensa VMs adicionais para máquinas monitoradas	~2 GB

Consumo resultante com a stack completa em execução: aproximadamente 3,1 GB.

3. Versões utilizadas

Componente	Versão	Observação
Ubuntu Server	24.04 LTS	host Docker
Docker Engine	estável (script oficial)	—

Componente	Versão	Observação
GLPI	10.0.26	API REST legada plenamente suportada
MySQL / MariaDB	8.0 / 10.11	banco do GLPI
Zabbix	7.0 LTS	previsto para a Fase 2
Wazuh	4.14.7	previsto para a Fase 3

4. Preparação do host

4.1 Endereçamento fixo

O endereço IP foi fixado via netplan. Em laboratório com DHCP, a troca de endereço invalida agentes, integrações e tokens configurados.

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses: [192.168.0.50/24]
      routes:
        - to: default
          via: 192.168.0.1      # <- seu gateway
      nameservers:
        addresses: [1.1.1.1, 8.8.8.8]
```

Figura 1 — Configuração de rede estática em /etc/netplan/50-cloud-init.yaml

4.2 Parâmetro de kernel exigido pelo Wazuh indexer

O Wazuh indexer é baseado em OpenSearch e exige o aumento do limite de áreas de memória virtual mapeadas. Sem esse ajuste, o container inicia e encerra em ciclo.

```
echo "vm.max_map_count=262144" | sudo tee /etc/sysctl.d/99-wazuh.conf
sudo sysctl -p /etc/sysctl.d/99-wazuh.conf
```

```
root@lab-server:~/lab-soc/glpi# sudo sysctl -p /etc/sysctl.d/99-wazuh.conf
vm.max_map_count = 262144
```

Figura 2 — Parâmetro aplicado e validado

5. Implantação do GLPI

O ambiente foi declarado em Docker Compose e versionado em Git, garantindo reprodutibilidade. As credenciais residem em arquivo .env, excluído do controle de versão.

```

1  name: glpi
2
3  services:
4    glpi:
5      image: glpi/glpi:10.0.26
6      container_name: glpi
7      restart: unless-stopped
8      env_file: .env
9      depends_on:
10       - db
11      ports:
12       - 8081:80
13
14  volumes:
15    - glpi_data:/var/glpi
16    - glpi_marketplace:/var/www/glpi/marketplace
17
18  db:
19    image: mysql:8.0
20    container_name: glpi-db
21    restart: unless-stopped
22    command:
23      - --character-set-server=utf8mb4
24      - --collation-server=utf8mb4_unicode_ci
25    environment:
26      MYSQL_ROOT_PASSWORD: ${GLPI_DB_ROOT_PASSWORD}
27      MYSQL_DATABASE: ${GLPI_DB_NAME}
28      MYSQL_USER: ${GLPI_DB_USER}
29      MYSQL_PASSWORD: ${GLPI_DB_PASSWORD}
30    volumes:
31      - glpi_db:/var/lib/mysql
32
33  volumes:
34    glpi_data:
35    glpi_marketplace:
36    glpi_db:

```

Figura 3 — docker-compose.yml da stack do GLPI

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
glpi	glpi/glpi:10.0.26	"/opt/glpi/entrypoint..."	glpi	14 minutes ago	Up 14 minutes	0.0.0.0:8081->80/tcp,
glpi-db	mysql:8.0	"docker-entrypoint.s..."	db	14 minutes ago	Up 14 minutes	3306/tcp, 33060/tcp

Figura 4 — Containers em execução

Figura 5 — Interface do GLPI acessível na porta 8081

6. Endurecimento inicial

6.1 Remoção das credenciais padrão

A instalação padrão do GLPI 10 cria quatro contas com credenciais públicas e amplamente documentadas. A senha da conta administrativa foi alterada e as demais contas foram desativadas.

Credenciais padrão do GLPI

Usuário	Senha	Perfil
glpi	glpi	Super-Admin
tech	tech	Técnico
normal	normal	Usuário
post-only	postonly	Só abre chamado

Figura 6 — Contas padrão da instalação

6.2 Conta de serviço com privilégio mínimo

Para a integração automatizada foi criada a conta `svc_integracao`, com perfil Technician restrito à entidade do laboratório, em vez de reutilizar a conta de super-administrador. Em caso de comprometimento do token, o alcance da ação é limitado e a rastreabilidade preservada.

Figura 7 — Conta de serviço e perfil atribuído

6.3 Estrutura de categorização

Foram criadas duas categorias raiz com subcategorias, permitindo separar, nos relatórios, os chamados originados pelo SIEM dos originados pelo monitoramento de infraestrutura.



Figura 8 — Árvore de categorias de chamados

7. Habilitação e validação da API REST

A API REST foi habilitada com autenticação por token externo e login por credenciais desabilitado. Foram gerados um token de aplicação (app_token) e um token de usuário vinculado à conta de serviço.

A validação foi realizada ainda nesta fase, deliberadamente, para que eventuais falhas de configuração não fossem descobertas apenas durante a implementação das integrações.

```
curl -sS -X GET "$GLPI/initSession" -H "Authorization: user_token $USER_TOKEN" -H "App-Token: $APP_TOKEN"
```

```
curl -sS -X POST "$GLPI/Ticket" -H "Session-Token: $SESSION" -H "App-Token: $APP_TOKEN" -d '{"input":{"name":"Chamado de teste via API","content":"Se voce esta lendo isso, a API funciona.,"type":1,"urgency":3}}' | jq
```

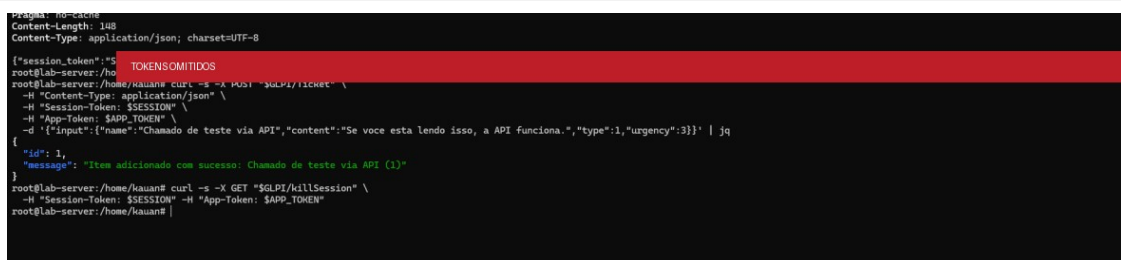


Figura 9 — initSession autenticado e chamado criado via API (tokens omitidos)

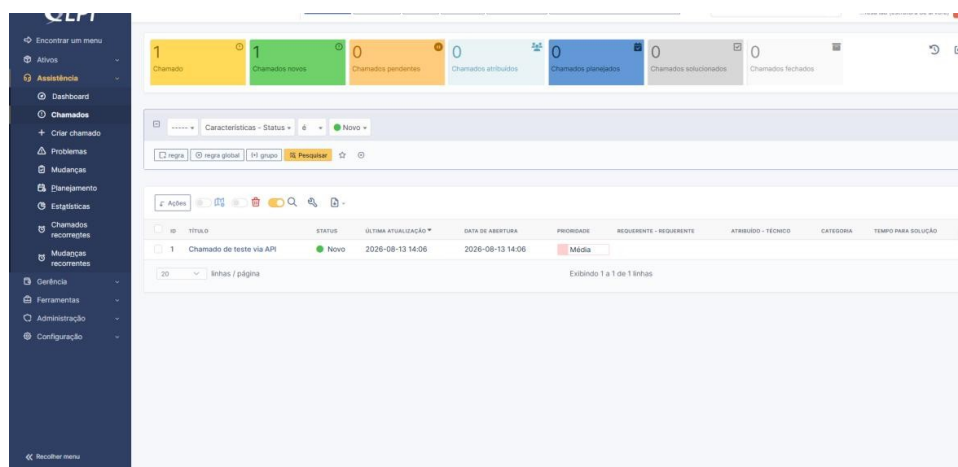


Figura 10 — Chamado registrado na interface do GLPI

8. Ocorrências e resolução

As falhas abaixo foram registradas durante a execução e representam a maior parte do tempo investido na fase.

Ocorrência	Causa raiz	Resolução
Autenticação SSH recusada com credencial correta	Divergência de layout de teclado entre o console da VM e o terminal do Windows, alterando a posição de caracteres especiais	Redefinição da senha com caracteres alfanuméricos e posterior adoção de autenticação por chave
ERROR 1049 (42000): Unknown database	Aninhamento incorreto de aspas separou o argumento -p da senha; o cliente MySQL interpretou o argumento seguinte como nome de banco	Execução do comando SQL em sessão interativa dentro do container
Requisições curl sem qualquer retorno	Variáveis de ambiente definidas antes de uma troca de usuário (sudo su) não são herdadas; a flag -s suprime a mensagem de erro	Redefinição das variáveis na sessão corrente e substituição de -s por -sS

9. Resultado da fase

- Host preparado, com endereçamento fixo e parâmetros de kernel ajustados
- GLPI 10.0.26 em execução, com schema instalado automaticamente
- Credenciais padrão removidas e conta de serviço criada com privilégio mínimo
- Estrutura de categorização definida para separar segurança de infraestrutura
- API REST habilitada e validada de ponta a ponta, com abertura de chamado confirmada

10. Próximas fases

Fase	Escopo
2	Zabbix 7.0 LTS: agentes, templates, triggers e monitoramento dos próprios containers
3	Wazuh 4.14.7: agentes Linux e Windows, regras de correlação, FIM e active response
4	Integrações: módulo integrator do Wazuh e media type webhook do Zabbix para a API do GLPI
5	Dashboards operacionais nas três ferramentas
6	Simulação de incidentes e validação da detecção

Observação: todos os tokens e credenciais exibidos durante a execução foram regenerados após a coleta das evidências.